## C++ Inheritance

---

### 1. Definition

Inheritance in C++ allows creation of a new class (derived class) from an existing class (base class) by extending it.
It promotes **code reusability** and establishes **is-a relationship**.

---

### 2. Reusability Approaches in C++

| Type | Relationship | Use Case |
|---|---|---|
| **Composition** | Has–a | When you need to use some functionalities of another class. |
| **Inheritance** | Is–a | When the new class represents a specialized version of another class. |

---

### 3. Members That Are *Not* Inherited

- Constructors
- Destructor
- Assignment operator (operator=)

---

### 4. Syntax

```
class Derived : <access – mode> Base
{
    // members
};
```

access-mode can be private, protected, or public.

---

### 5. Inheritance Access Mode Hierarchy

```
Inheritance Mode
│
├──→ Private Inheritance
│       ├── Parent Private    → Not Accessible
│       ├── Parent Protected  → Private in Child
│       └── Parent Public     → Private in Child
│
├──→ Protected Inheritance
│       ├── Parent Private    → Not Accessible
│       ├── Parent Protected  → Protected in Child
│       └── Parent Public     → Protected in Child
│
└──→ Public Inheritance
        ├── Parent Private    → Not Accessible
        ├── Parent Protected  → Protected in Child
        └── Parent Public     → Public in Child
```

---

### 6. Key Accessibility Rules

1. **Private** and **Protected** differ **only in inheritance**.
    - private → not accessible in child.
    - protected → accessible in child (depending on inheritance mode).
2. **Private members of base** are never accessible in derived class.
3. **Protected** members of base are inherited but not accessible outside the class hierarchy.

---

### 7. Promoting Base Class Members

You can **make a base class member accessible (public)** in derived class using the scope declaration:

```
class Derived : private Base
{
public:
    Base::someFunction;   // makes inherited base method accessible publicly
};
```

---

### 8. Constructor & Destructor Behavior

#### a) Order of Constructor/Destructor Execution

- **Constructors** are called **top-down** (Base → Derived).
- **Destructors** are called **bottom-up** (Derived → Base).

---

#### b) Base Constructor Invocation

If base class has a **default constructor**, it is invoked automatically.
If not, the derived class must **explicitly invoke** base's parameterized constructor.

**Syntax:**

```
Derived() : Base(args)
{
    // derived constructor body
}
```

If not provided → **compilation error**.

---

#### c) Private / Protected Base Constructors

| Parent Constructor Access | Child Instantiation | Parent Instantiation |
|---|---|---|
| **private** | ❌ Not possible | ❌ Not possible |
| **protected** | ✅ Possible | ❌ Not possible |
| **public** | ✅ Possible | ✅ Possible |

**Example Case**

```
class Base {
protected:
    Base() {}  // protected constructor
};

class Derived : public Base {
public:
    Derived() {}  // valid
};

// Base b;  // ❌ not allowed
// Derived d;  // ✅ allowed
```

**Examples:**

- Classes like istream, ostream have **protected constructors**.
  → Cannot instantiate directly, but can instantiate derived (ifstream, ofstream).

---

### 9. Constructor Inheritance Constraints

#### If Parent Has No Default Constructor

- Compiler will generate an error if the parent lacks a default constructor.
- To resolve:
  - Provide a **default constructor** in parent
    **or**
  - **Explicitly invoke** parent's parameterized constructor from child.

```
class Child : public Parent {
public:
    Child() : Parent(10) {} // explicit invocation
};
```

**Note:** Every constructor of the child must invoke the parent's constructor explicitly if no default exists.

---

### 11. Types of Inheritance in C++

| Type | Description |
|---|---|
| **Single** | One base → one derived |
| **Multilevel** | Derived acts as base for another |
| **Hierarchical** | One base → multiple derived |
| **Multiple** | Derived from multiple bases |
| **Hybrid** | Combination of the above |

---

**12. Multiple Inheritance Rules**

- Constructors are **invoked in order of inheritance declaration**, not in the order of base list.
  Example:

    1. class Derived : Base1, public Base2, protected Base3
    2. Constructors called: Base1 → Base2 → Base3 → Derived
    3. Destructors are called in **reverse** order.
- **Ambiguity in Multiple Inheritance**
    - If two base classes contain **functions with same name**,
      accessing them from child class without qualification causes **ambiguity**.

      ```
      base1::disp();
      base2::disp();
      ```
- Must use **scope resolution operator** to specify which base function to call.

must be explicitly used to resolve ambiguity.

---

**13. Access Control in Multiple Inheritance**

Each base can be inherited with a **different access specifier**:

```
class Derived : public Base1, protected Base2, private Base3
```

Each inheritance affects accessibility independently.

---

**11. Friend Function and Inheritance**

- Friendship **is not inherited**.
- A **friend function of base** is **not automatically a friend of derived**.

**Syntax:**

```
friend void fun(Base&, Derived&);
```

---

**15. Protected Accessibility Demonstration**

- private and protected behave identically **except during inheritance**.
- protected allows visibility in derived classes.

**Syntax:**

```
class Base {
protected:
    int data;
};
class Derived : public Base {
    void show() { cout << data; } // accessible
};
```

---

### 16. Summary of C++-Specific Points

| Concept | Java | C++ Behavior |
|---|---|---|
| **Access Modes** | extends only (public) | private, protected, public inheritance |
| **Multiple Inheritance** | Not allowed | Allowed, with ambiguity rules |
| **Constructor Chaining** | Always implicit using super keyword | Must be explicit if base has no default constructor |
| **Base Constructor Access** | Always public | Can be private/protected (affects instantiation) |
| **Friend Functions** | N/A | Exist, not inherited |
| **Member Promotion** | N/A | Can re-expose base members using Base::member |
| **Virtual Base Classes** | N/A | Used to solve diamond problem |
| **Destructor Order** | Automatic | Must ensure virtual destructors when deleting polymorphically |

---

### 17. Syntax Summary (No Full Code)

```
// Single inheritance
class Derived : public Base {};

// Multilevel
class Derived2 : public Derived1 {};

// Multiple inheritance
class Derived : public Base1, protected Base2 {};

// Hierarchical
class Derived1 : public Base {};
class Derived2 : protected Base {};

// Explicit base constructor call
Derived() : Base(args) {}

// Member promotion
class Derived : private Base {
public:
    Base::functionName;
};

// Friend in inheritance
friend void fun(Base&, Derived&);
```

---

### Diamond Problem in C++

### 18. Definition

The **Diamond Problem** occurs in **multiple inheritance** when a class inherits from two parent classes that both derive from a common base class.

This results in **two separate copies** of the base class being inherited into the most derived class, creating **ambiguity** and **duplicate subobjects**.

**In short:**

The **Diamond Problem** arises due to multiple copies of a common base class in multiple inheritance.
**Virtual inheritance** ensures only **one shared instance** of that base class exists, thus eliminating ambiguity and redundancy.

---

**19. Example Scenario**

- Gp → Base (Grandparent) class
- P1 and P2 → Both inherit from Gp
- child → Inherits from both P1 and P2

So, child indirectly inherits **two copies** of Gp:
one through P1 and one through P2.

---

**20. Problem Explanation**

When you create an object of child, internally two separate Gp subobjects are created —
child → P1 → Gp and child → P2 → Gp.

Thus, if you try to call:

c.disp1();

The compiler will report **ambiguity**, because it does not know whether to use Gp's disp1() inherited through P1 or through P2.

This is the **Diamond Problem**.

---

**21. Temporary (Manual) Solution**

You can explicitly qualify which path to take:

c.P1::disp1();

c.P2::disp1();

However, this approach is **not scalable** and **defeats the purpose of inheritance**, as it requires manual disambiguation each time.

---

**22. Permanent Solution — Virtual Inheritance**

To ensure that **only one shared instance** of the common base (Gp) exists, you declare it as **virtual** in both intermediate classes:

class P1 : virtual public Gp { };

class P2 : virtual public Gp { };

Now, when child inherits from both P1 and P2, the compiler ensures that **only one shared copy** of Gp exists inside the child object.

---

**23. Behavior after Virtual Inheritance**

- Only **one** Gp constructor and **one** Gp destructor** will execute.
- Ambiguity is resolved:
  You can now directly call c.disp1() without qualification.
- The object layout now contains a **single virtual base subobject** for Gp.

---

**24. Summary Table**

| Case | Declaration | Result |
|---|---|---|
| **Without virtual** | class P1 : public Gpclass P2 : public Gp | Two separate copies of Gp → Ambiguity |
| **With virtual** | class P1 : virtual public Gpclass P2 : virtual public Gp | One shared copy of Gp → No ambiguity |

---

**25. Key Points**

- **Virtual inheritance** ensures a base class is shared among derived classes.
- It prevents **multiple base subobjects** from being created.
- Used mainly to solve the **Diamond Problem** in multiple inheritance.
- Order of constructor and destructor calls changes — virtual base is constructed **first** and destroyed **last**.

---

**26. Syntax Highlights (for Quick Reference)**

| Concept | Syntax |
|---|---|
| **Basic Inheritance** | `class Child : public Parent {};` |
| **Explicit Parent Constructor Call** | `Child() : Parent(args) {}` |
| **Expose Parent Method** | `Parent::method;` |
| **Friend Function** | `friend void fun(Base&, Derived&);` |
| **Protected Inheritance Syntax** | `class Sub : protected Base {};` |
| **Multiple Inheritance Syntax** | `class Sub : public Base1, protected Base2 {};` |
| **Ambiguity Resolution** | `Base1::function();` |

---

## Relationships

### 1. “is-a” and “has-a” Relationships

- **is-a relationship** represents **inheritance** in C++.
  It indicates that a derived class is a specialized version of a base class.
  Example:

```
class Car : public Vehicle {}; // Car is-a Vehicle
```

  **has-a relationship** represents **composition or aggregation** (object containment).
  It indicates that one class contains an object of another class.
  Example:

```
class Engine { };
class Car {
  Engine e; // Car has-a Engine
};
```

---

### 2. Order of Constructor and Destructor Calls

When both inheritance (**is-a**) and containment (**has-a**) are used:

**Constructor Call Order:**

Base class constructor (is-a part) → (has-a part) → Derived class

**Destructor Call Order:**
The reverse of constructors:

Derived class → (has-a part) → Base class constructor (is-a part)

---

### 3. Parameterized Constructors

In case of **inheritance (is-a)**, the **base class constructor** must be called explicitly from the derived class constructor **using an initializer list**:

```
class Base {
public:
    Base(int x) {}
};

class Derived : public Base {
public:
    Derived(int a) : Base(a) {}  // is-a → called by class name
};
```

In case of **composition (has-a)**, the **contained object’s constructor** is also called through the **initializer list**, but by **using the member object’s name**:

```
class Engine {
public:
    Engine(int power) {}
};

class Car {
    Engine e;
public:
```

```
    Car(int p) : e(p) {}  // has-a → called by object name
};
```

---

### 4. Aggregation

- **Aggregation** means one class has a reference or pointer to another class's object.
- The **contained object is created outside** and passed to the class (usually via a method or constructor).
- When the container class is destroyed, the aggregated object **continues to exist independently**.

Example:

```
class Engine {};
class Car {
    Engine* e;  // aggregation via pointer
public:
    void setEngine(Engine* eng) { e = eng; }  // passing object externally
};
```

Here, destroying Car does **not** destroy the Engine.

---

### 5. Composition

- **Composition** means one class **creates and owns** the object of another class inside it.
- When the containing class is destroyed, the **contained object is also destroyed automatically**.

Example:

```
class Engine {};
class Car {
        Engine e;  // composition
};
```

Here, when Car goes out of scope, its Engine object is also destroyed.

---

## Redefinition (Function Hiding) in C++

---

#### 1. Definition

When a **derived class** defines a function having the **same name** as one in its **base class**, the **base class function gets hidden** in the derived scope.
This is called **function redefinition** or **name hiding** (not overriding unless virtual is used).

---

#### 2. Case 1 — Same Function Signature (True Redefinition)

```
class Base {
public:
    void disp() { cout << "Base disp\n"; }
};
class Sub : public Base {
public:
```

```
    void disp() { cout << "Sub disp\n"; }
};
int main() {
    Sub s;
    s.disp();        // Sub disp
    s.Base::disp();  // Base disp
}
```

**Concept:**

Derived function **redefines** the base version (same name & parameters).

Base function is hidden but can be accessed using Base::.

---

### 3. Case 2 — Different Function Signature (Name Hiding)

```
class Base { public: void disp(int k){ cout<<"Base disp\n"; } };
class Sub : public Base { public: void disp(){ cout<<"Sub disp\n"; } };
int main(){
    Sub s;
    s.disp();         // Sub disp
    // s.disp(10);    // Error: Base version hidden
    s.Base::disp(10); // Base disp
}
```

**Concept:**

Even with **different parameters**, the derived class **hides all overloads** of that name from the base class.

---

### 4. Case 3 — Restoring Hidden Base Function

```
class Sub : public Base {
public:
    using Base::disp;   // re-expose base versions
    void disp() { cout << "Sub disp\n"; }
};
```

**Concept:**

using Base::disp; allows **both base and derived** versions to coexist.

---

### 5. Key Rules Summary

| Case | Description | Effect | Access Base |
|---|---|---|---|
| **Same name + same params** | Redefinition | Derived hides base | s.Base::disp() |
| **Same name + diff params** | Name hiding | All base overloads hidden | s.Base::disp() |
| **Use using Base::func;** | Restores base functions | Both accessible | Not needed |

---